

# Object Tracking: Lucas-Kanade & Matthew-Baker

## 1. Introduction

The idea here is to draw a box around something in the first frame of a video, and the tracker keeps that box on the object as it moves. To do that, each frame we solve for a warp that lines the current frame back up with the template saved at the start. I implemented two versions. The first is the standard forward-additive Lucas-Kanade tracker, which estimates a small motion between each pair of frames. The second is the Matthew-Baker method, which does essentially the same thing but runs much faster because it computes its Jacobian and Hessian once from the fixed template and reuses them the whole way through. I tested both on a car sequence and the MB tracker on a runway sequence, and the write-up also covers two short theory questions.

## 2. Methods

### 2.1 Theory

#### Q1.1

The affine warp moves a pixel  $x = [u \ v \ 1]^T$  with the matrix

$W(p) =$

|         |         |       |
|---------|---------|-------|
| $1+p_1$ | $p_3$   | $p_5$ |
| $p_2$   | $1+p_4$ | $p_6$ |
| 0       | 0       | 1     |

so the two output coordinates are  $u' = (1+p_1)u + p_3 v + p_5$  and  $v' = p_2 u + (1+p_4)v + p_6$ . The Jacobian  $\partial W / \partial p$  just collects the partial derivatives of these with respect to each parameter, which works out to

$\partial W / \partial p =$

|     |     |     |     |   |   |
|-----|-----|-----|-----|---|---|
| $u$ | 0   | $v$ | 0   | 1 | 0 |
| 0   | $u$ | 0   | $v$ | 0 | 1 |

#### Q1.2

With  $n$  template pixels,  $m$  image pixels, and  $p$  parameters: initialization builds  $J = \nabla T \cdot \partial W / \partial p$  in  $O(np)$ , forms the Hessian  $H = J^T J$  in  $O(np^2)$  (the dominant term), and inverts it in  $O(p^3)$ , so it is  $O(np^2 + p^3)$  overall, and notice  $m$  never enters, since init only touches the template. Each runtime iteration then costs only  $O(np + p^3)$ : warp, error,  $J^T \cdot \text{error}$ , multiply by the precomputed  $H^{-1}$ , compose the update.

The contrast with plain Lucas-Kanade is the whole point. In LK the gradient and Jacobian are evaluated on the current image at the current  $p$ , so the Hessian has to be rebuilt and re-inverted every iteration,  $O(np^2 + p^3)$  each time. Matthew-Baker pays that  $O(np^2)$  once instead of every step, and I saw this directly: the MB tracker ran noticeably faster than LK on the same clip.

### 2.2 Lucas-Kanade tracker (Q2.1)

My LucasKanade function takes the template from the previous frame and finds the  $(u, v)$  that best lines the current frame up with it. After converting to grayscale doubles and cropping the template,

it starts from the identity warp and iterates: build  $W$  from the current parameters, warp the current frame and its gradients, crop to the template, form the error  $T - I(W)$ , assemble  $A = \nabla I \cdot \partial W / \partial p$ , and solve the normal equations for  $\Delta p$ . It stops once  $\Delta p$  is small, and since the demo only needs translation I read  $u$  and  $v$  off  $p_5$  and  $p_6$  at the end.

The one thing that genuinely tripped me up was the provided `warpH` function. It samples the image at  $H^{-1} \cdot x$  rather than  $H \cdot x$ , so to actually get  $I(W(x;p))$ , (the current frame at the warped coordinates) I had to pass  $\text{inv}(W)$  into it, not  $W$ . I only trusted this after testing it on an image shifted by a known amount: with  $\text{inv}(W)$  the tracker recovers the exact shift in a few iterations, and the other way it just diverges.

### 2.3 Matthew-Baker initialization (Q2.2)

`initAffineMBTracker` precomputes the two things that stay fixed for the whole sequence. It takes the template gradient, builds the affine Jacobian, forms the steepest-descent images  $J = \nabla T \cdot \partial W / \partial p$ , computes  $H = J^T J$ , and inverts it. Since none of this depends on  $p$  or on the current frame, it only has to happen once.

### 2.4 Matthew-Baker tracker (Q2.3)

`affineMBTracker` does the per-frame work. Starting from the previous frame's warp, each iteration warps the current frame by  $\text{inv}(W)$ , takes the error  $I(W) - T$ , solves  $\Delta p = H^{-1} J^T b$  with the precomputed matrices, and applies it the inverse-compositional way by composing the inverse onto  $W$  ( $W \leftarrow W \cdot W(\Delta p)^{-1}$ ). I checked the whole loop on synthetic data first and it recovers both a known translation and a known affine almost exactly. For display I push the template centre through the final warp to place the box, and each frame warm-starts from the last one so the updates stay small even as the total motion builds up.

A few smaller things were worth getting right. Both trackers pass  $\text{inv}(W)$  to `warpH` for the reason above. On the runway sequence the affine warp can run away and go singular, which makes  $\text{inv}(W)$  non-finite and crashes `warpH`, so I added a guard that falls back to the last good warp when that happens, the tracker degrades instead of dying. And because `getframe` returned different frame sizes on my display, I resize every captured frame to a fixed size before writing the video.

## 3. Results

### 3.1 Lucas-Kanade on the car (Q2.4)

I initialized the LK box at `[160 120 140 140]` on the back of the SUV. It holds the car for about the first half of the clip, then slides off to the left and never recovers, ending up pinned at the edge of the frame.



Figure 1: LK box locked onto the car early in the sequence.



Figure 2: Later on, the box has drifted left off the car onto the roadside trees.

This is what I'd expect from plain LK, with a few reasons stacked on top of each other. The tracker is translation-only and rebuilds its template every frame, so small errors just accumulate. My box also sits half on the car and half on the background, and as the camera vehicle drives forward the roadside trees sweep left faster than the car ahead, so the strong tree texture slowly drags the box off target. The SUV itself mostly grows rather than moves, and translation can't follow scale, so there isn't much holding the box in place. In short, LK wants a well-textured, rigid target moving by translation, and it breaks here because of clutter, scale, and drift.

### 3.2 Matthew-Baker on the car (Q2.4)

For MB I used [135 107 210 170], centred on the car. It ran clearly faster than LK. The Q1.2 result showing up in practice and held the car through most of the clip, well past where LK failed. It finally breaks when the car goes under a bridge.

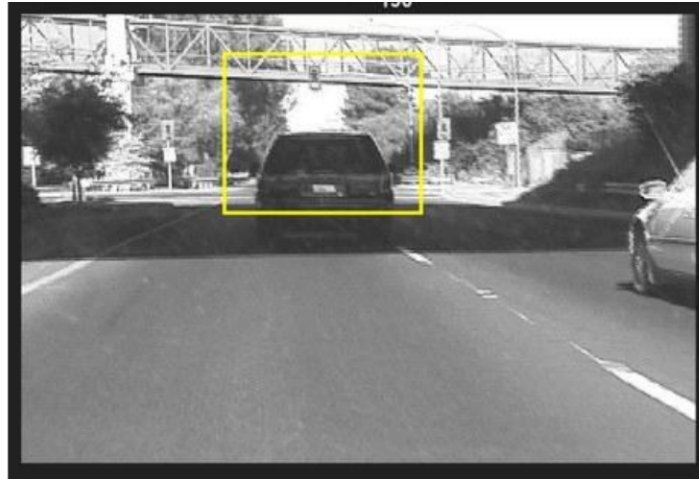


Figure 3: MB tracker as the car approaches the shadow under the bridge; the fit starts to slip.



Figure 4: The box gets captured by strong side edges (lane arrow / guard rail) as the car, now darker and smaller, pulls away.

I believe the main culprit is the lighting. The tracker compares raw pixels against a template frozen from the first sunny frame, which quietly assumes the car's brightness never changes. Under the bridge the whole patch drops into shadow, so the error is large everywhere even when the box is right, and the solver chases that and slides off. Two things make it worse: the car is shrinking as it pulls away, so the big bright template fits worse and worse, and once the box is loose it gets grabbed by strong nearby edges like the bridge frame and the white lane arrow. After the bridge the lighting recovers and the box starts creeping back toward the car, but the clip ends first. So the templates that work are the ones that stay on a textured, rigid part of the object and don't change much from the first frame, and the clearest breakdown is the lighting change under the bridge.

### 3.3 Matthew-Baker on the runway (Q2.5)

I ran MB on the runway too, starting on the painted "35" since it's the only high-contrast, textured thing in an otherwise dark frame. It holds the marking at first, then distorts and collapses into the top-left corner as the runway comes up, and the warp diverges.



Figure 5: After divergence the affine box has collapsed to a small region in the corner, having lost the "35".

This fails differently from the car. As the aircraft drops, the "35" grows and its perspective shifts, so the fixed first-frame template matches worse and worse, and since everything around it is dark and nearly textureless, the Hessian is poorly conditioned and can't hold the fit steady. Once the match slips, the affine scale parameters run away, the warp matrix shrinks toward singular (that's the box collapsing in the corner), its condition number blows up, and  $\text{inv}(W)$  eventually goes non-finite. That's what would crash warph; my guard catches it and freezes the last good warp so the run finishes.

#### 4. Conclusion

I implemented and compared the two trackers, derived the affine Jacobian, and worked out why Matthew-Baker is faster, it pays the  $O(np^2)$  Hessian cost once instead of every iteration, which I could see in the run times. On the car, LK held the target for about half the clip before drifting, limited by its translation-only model and frame-to-frame drift, while MB ran faster and lasted most of the clip before the bridge lighting broke it. On the runway, MB followed the "35" until the growing scale and low-texture background made the affine warp diverge. The limitations that showed up are the familiar ones: both trackers lean on brightness constancy, both struggle with large scale or perspective change against a fixed template, and LK specifically can't model scale and accumulates drift.